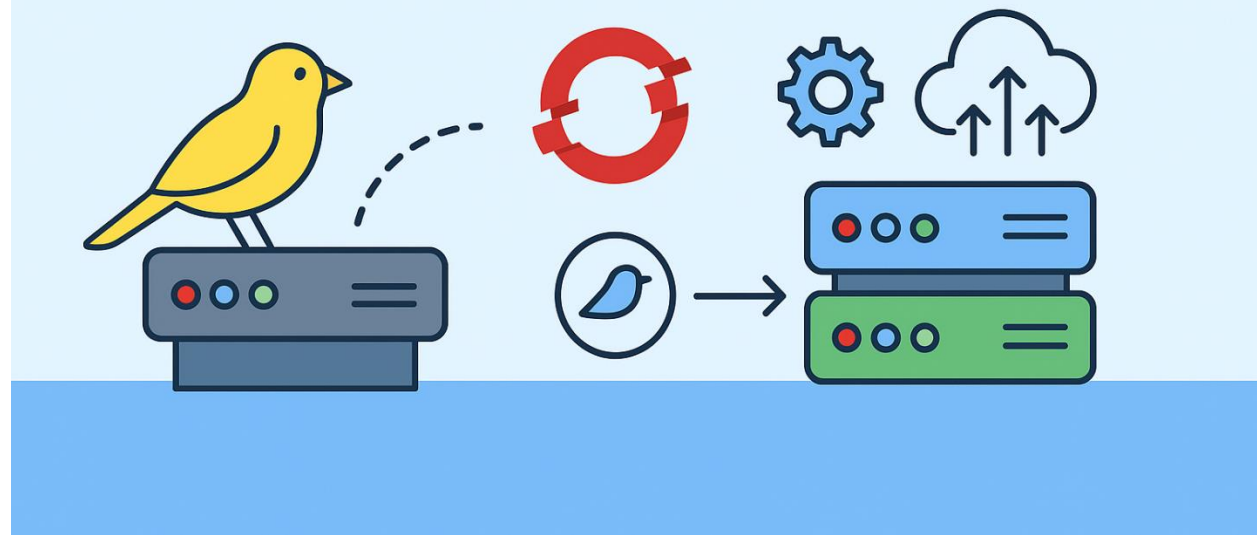


For more details, please visit <https://assistedlabs.com>

Canary and Blue-Green Deployments in OpenShift

Canary and Blue-Green Deployments in OpenShift



Introduction

In the world of modern application delivery, minimizing downtime and reducing risk during deployments is crucial.

Two popular deployment strategies that help achieve these goals are **Canary Deployments** and **Blue-Green Deployments**.

OpenShift, a Kubernetes-based container platform, supports both strategies natively, leveraging its **deployment configurations**, **routes**, and **service management** features.

These strategies enable teams to:

- Release new versions with minimal disruption.
- Test new features in production with controlled exposure.

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>

For more details, please visit <https://assistedlabs.com>

- Quickly roll back in case of failures.
-

Overview of Deployment Strategies

Canary Deployment

A **Canary Deployment** gradually rolls out a new version to a small subset of users before releasing it to everyone. This allows teams to test the application in the real-world environment and monitor performance.

Key Characteristics:

- Only a small percentage of traffic is directed to the new version initially.
- Gradual traffic shift over time.
- Easier to detect and fix issues early.

When to Use:

- When changes are small but may impact critical functionality.
 - For performance and compatibility testing in production.
-

Blue-Green Deployment

A **Blue-Green Deployment** maintains two identical environments:

- **Blue** – the current running version (live production).
- **Green** – the new version ready to replace Blue.

When the new version is ready, traffic is switched from Blue to Green instantly. If issues arise, traffic can quickly be reverted to Blue.

Key Characteristics:

- Two complete environments.
- Instant switch with minimal downtime.
- Simple rollback mechanism.

When to Use:

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>

For more details, please visit <https://assistedlabs.com>

- For critical production releases where downtime must be avoided.
- When infrastructure cost is acceptable for duplicate environments.

Canary Deployment in OpenShift

How It Works in OpenShift

OpenShift enables Canary Deployments using:

- **Multiple Deployment Configurations** or **Deployments** for old and new versions.
- **Route weights** to control traffic distribution between versions.
- **Horizontal Pod Autoscaling** for scaling based on load.

Basic Flow:

1. Deploy the stable version (v1).
2. Deploy the canary version (v2) alongside v1.
3. Use route weight to send a small percentage of traffic to v2 (e.g., 10%).
4. Monitor metrics and logs.
5. Gradually increase traffic until v2 handles 100%.
6. Remove v1 after confirming stability.

Example Setup

```
# Create the stable version
oc new-app myapp:v1 --name=myapp-v1

# Create the canary version
oc new-app myapp:v2 --name=myapp-v2

# Expose both as services
oc expose svc/myapp-v1
oc expose svc/myapp-v2

# Create a single route with weighted traffic
oc create route edge myapp --service=myapp-v1 --port=8080
oc set route-backends myapp myapp-v1=90 myapp-v2=10
```

Benefits and Risks

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>

ASSISTED LABS

For more details, please visit <https://assistedlabs.com>

Benefits:

- Reduced risk for production failures.
- Real-time performance testing.
- Gradual and controlled rollout.

Risks:

- Requires monitoring and traffic routing control.
- Slower release process compared to instant switches.

Blue-Green Deployment in OpenShift

How It Works in OpenShift

Blue-Green in OpenShift is typically implemented by:

- Deploying both Blue and Green environments in parallel.
- Using a **single route** that points to the active environment.
- Switching route targets instantly during release.

Basic Flow:

1. Deploy Blue (v1) and set it as live.
2. Deploy Green (v2) in parallel.
3. Run tests on Green in isolation.
4. Switch the route to point to Green instantly.
5. Roll back to Blue if problems occur.

Example Setup

```
# Deploy Blue (current)
oc new-app myapp:v1 --name=myapp-blue
oc expose svc/myapp-blue --name=myapp

# Deploy Green (new)
oc new-app myapp:v2 --name=myapp-green
oc expose svc/myapp-green

# Switch traffic from Blue to Green
oc set route-backends myapp myapp-green=100 myapp-blue=0
```

ASSISTED LABS

For more details, please visit <https://assistedlabs.com>

Benefits and Risks

Benefits:

- Minimal downtime during switch.
- Simple rollback process.
- Predictable release cycle.

Risks:

- Higher infrastructure cost (duplicate environments).
- Requires proper synchronization of databases and state.

Best Practices for Both Strategies

1. **Automate Deployments**
 - Use OpenShift Pipelines (Tekton) or Jenkins for automated builds, tests, and deployments.
2. **Monitor Performance**
 - Use OpenShift Monitoring (Prometheus, Grafana) to track application health.
3. **Test Before Traffic Shift**
 - Run automated tests and manual checks before exposing new versions.
4. **Plan Rollback**
 - Keep rollback scripts ready to revert quickly.
5. **Manage Configuration**
 - Store all deployment configurations in Git for version control.

Choosing Between Canary and Blue-Green

Factor	Canary Deployment	Blue-Green Deployment
Downtime	None (gradual rollout)	None (instant switch)
Risk Level	Low – gradual exposure	Low – quick rollback
Cost	Lower – fewer duplicate resources	Higher – full duplicate setup
Speed	Slower rollout	Faster switch
Use Case	Feature validation in production	Critical releases, large updates

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>

For more details, please visit <https://assistedlabs.com>

Conclusion

Both **Canary** and **Blue-Green Deployments** are powerful strategies in OpenShift for delivering applications with minimal risk and downtime.

- **Canary** is ideal for gradual feature rollouts with real-user testing.
- **Blue-Green** suits instant switches between versions, especially for mission-critical systems.

By leveraging OpenShift's routing, scaling, and automation capabilities, organizations can implement these strategies effectively to achieve safe, reliable, and fast application delivery.

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>